

3. An Informal Introduction to Python

In the following examples, input and output are distinguished by the presence or absence of prompts (`*>>>*` and `*...*`): to repeat the example, you must type everything after the prompt, when the prompt appears; lines that do not begin with a prompt are output from the interpreter. Note that a secondary prompt on a line by itself in an example means you must type a blank line; this is used to end a multi-line command.

Many of the examples in this manual, even those entered at the interactive prompt, include comments. Comments in Python start with the hash character, `"#"`, and extend to the end of the physical line. A comment may appear at the start of a line or following whitespace or code, but not within a string literal. A hash character within a string literal is just a hash character. Since comments are to clarify code and are not interpreted by Python, they may be omitted when typing in examples.

Some examples:

```
# this is the first comment
```

```
spam = 1 # and this is the second comment
```

```
    # ... and now a third!
```

```
text = "# This is not a comment because it's inside quotes."
```

with a fractional part (e.g. "5.0", "1.6") have type "float". We will see more about numeric types later in the tutorial.

Division ("/") always returns a float. To do *floor division* and get an integer result you can use the "//" operator; to calculate the remainder you can use "%":

```
>>> 17 / 3 # classic division returns a float
5.666666666666667
>>>
>>> 17 // 3 # floor division discards the fractional part
5
>>> 17 % 3 # the % operator returns the remainder of the division
2
>>> 5 * 3 + 2 # floored quotient * divisor + remainder
17
```

With Python, it is possible to use the "**" operator to calculate powers [1]:

```
>>> 5 ** 2 # 5 squared
25
>>> 2 ** 7 # 2 to the power of 7
128
```

The equal sign ("=") is used to assign a value to a variable.

Afterwards, no result is displayed before the next interactive prompt:

```
>>> price * tax
```

```
12.5625
```

```
>>> price + _
```

```
113.0625
```

```
>>> round(_, 2)
```

```
113.06
```

This variable should be treated as read-only by the user. Don't explicitly assign a value to it --- you would create an independent local variable with the same name masking the built-in variable with its magic behavior.

In addition to "int" and "float", Python supports other types of numbers, such as "Decimal" and "Fraction". Python also has built-in support for complex numbers, and uses the "j" or "J" suffix to indicate the imaginary part (e.g. "3+5j").

3.1.2. Text

Python can manipulate text (represented by type "str", so-called "strings") as well as numbers. This includes characters "!", words "rabbit", names "Paris", sentences "Got your back.", etc. "Yay! :)". They can be enclosed in single quotes ("...") or double quotes ("...") with the same result [2].

```
>>> s # without print(), special characters are included in the string
```

```
'First line.\nSecond line.'
```

```
>>> print(s) # with print(), special characters are interpreted, so \n produces new line
```

```
First line.
```

```
Second line.
```

If you don't want characters prefaced by "\" to be interpreted as special characters, you can use *raw strings* by adding an "r" before the first quote:

```
>>> print('C:\this\name') # here \t means tab, \n means newline
```

```
C:  his
```

```
ame
```

```
>>> print(r'C:\this\name') # note the r before the quote
```

```
C:\this\name
```

There is one subtle aspect to raw strings: a raw string may not end in an odd number of "\" characters; see the FAQ entry for more information and workarounds.

String literals can span multiple lines. One way is using triple-quotes: `"""..."""` or `'''...'''`. End-of-line characters are automatically included in the string, but it's possible to prevent this by adding a "\" at the end of the line. In the following example, the initial newline is not included:

```
>>> print("""\
```

```
... 'to have them joined together.')
```

```
>>> text
```

```
'Put several strings within parentheses to have them joined together.'
```

This only works with two literals though, not with variables or expressions:

```
>>> prefix = 'Py'
```

```
>>> prefix 'thon' # can't concatenate a variable and a string literal
```

```
File "<stdin>", line 1
```

```
prefix 'thon'
```

```
^^^^^^
```

```
SyntaxError: invalid syntax
```

```
>>> ('un' * 3) 'ium'
```

```
File "<stdin>", line 1
```

```
('un' * 3) 'ium'
```

```
^^^^^^
```

```
SyntaxError: invalid syntax
```

If you want to concatenate variables or a variable and a literal, use

"+":

```
>>> prefix + 'thon'
```

```
'Python'
```

Strings can be *indexed* (subscripted), with the first character having index 0. There is no separate character type; a character is

'tho'

Slice indices have useful defaults; an omitted first index defaults to zero, an omitted second index defaults to the size of the string being sliced.

```
>>> word[:2] # character from the beginning to position 2 (excluded)
```

'Py'

```
>>> word[4:] # characters from position 4 (included) to the end
```

'on'

```
>>> word[-2:] # characters from the second-last (included) to the end
```

'on'

Note how the start is always included, and the end always excluded.

This makes sure that "s[:i] + s[i:]" is always equal to "s":

```
>>> word[:2] + word[2:]
```

'Python'

```
>>> word[:4] + word[4:]
```

'Python'

One way to remember how slices work is to think of the indices as pointing *between* characters, with the left edge of the first character numbered 0. Then the right edge of the last character of a string of *n* characters has index *n*, for example:

+---+---+---+---+---+---+

"

Python strings cannot be changed --- they are **immutable**. Therefore, assigning to an indexed position in the string results in an error:

```
>>> word[0] = 'J'
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

TypeError: 'str' object does not support item assignment

```
>>> word[2:] = 'py'
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

TypeError: 'str' object does not support item assignment

If you need a different string, you should create a new one:

```
>>> 'J' + word[1:]
```

```
'Jython'
```

```
>>> word[:2] + 'py'
```

```
'Pypy'
```

The built-in function "len()" returns the length of a string:

```
>>> s = 'supercalifragilisticexpialidocious'
```

```
>>> len(s)
```

34

Lists might contain items of different types, but usually the items all have the same type.

```
>>> squares = [1, 4, 9, 16, 25]
>>> squares
[1, 4, 9, 16, 25]
```

Like strings (and all other built-in *sequence* types), lists can be indexed and sliced:

```
>>> squares[0] # indexing returns the item
1
>>> squares[-1]
25
>>> squares[-3:] # slicing returns a new list
[9, 16, 25]
```

Lists also support operations like concatenation:

```
>>> squares + [36, 49, 64, 81, 100]
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Unlike strings, which are *immutable*, lists are a *mutable* type, i.e. it is possible to change their content:

```
>>> cubes = [1, 8, 27, 65, 125] # something's wrong here
>>> 4 ** 3 # the cube of 4 is 64, not 65!
```

elements. This means that the following slice returns a shallow copy of the list:

```
>>> correct_rgba = rgba[:]
>>> correct_rgba[-1] = "Alpha"
>>> correct_rgba
["Red", "Green", "Blue", "Alpha"]
>>> rgba
["Red", "Green", "Blue", "Alph"]
```

Assignment to slices is also possible, and this can even change the size of the list or clear it entirely:

```
>>> letters = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
>>> letters
['a', 'b', 'c', 'd', 'e', 'f', 'g']
>>> # replace some values
>>> letters[2:5] = ['C', 'D', 'E']
>>> letters
['a', 'b', 'C', 'D', 'E', 'f', 'g']
>>> # now remove them
>>> letters[2:5] = []
>>> letters
['a', 'b', 'f', 'g']
>>> # clear the list by replacing all the elements with an empty list
>>> letters[:] = []
>>> letters
```

sequence of the Fibonacci series as follows:

```
>>> # Fibonacci series:
>>> # the sum of two elements defines the next
>>> a, b = 0, 1
>>> while a < 10:
...     print(a)
...     a, b = b, a+b
...
0
1
1
2
3
5
8
```

This example introduces several new features.

* The first line contains a **multiple assignment**: the variables "a" and "b" simultaneously get the new values 0 and 1. On the last line this is used again, demonstrating that the expressions on the right-hand side are all evaluated first before any of the assignments take place. The right-hand side expressions are evaluated from the left to the right.

* The "while" loop executes as long as the condition (here: "a < 10")

```
>>> print('The value of i is', i)
```

The value of i is 65536

The keyword argument `*end*` can be used to avoid the newline after the output, or end the output with a different string:

```
>>> a, b = 0, 1
```

```
>>> while a < 1000:
```

```
...     print(a, end=',')
```

```
...     a, b = b, a+b
```

```
...
```

```
0,1,1,2,3,5,8,13,21,34,55,89,144,233,377,610,987,
```

-[Footnotes]-

[1] Since `***` has higher precedence than `-`, `"-3**2"` will be interpreted as `-(3**2)` and thus result in `"-9"`. To avoid this and get `"9"`, you can use `"(-3)**2"`.

[2] Unlike other languages, special characters such as `"\n"` have the same meaning with both single (`"..."`) and double (`"..."`) quotes. The only difference between the two is that within single quotes you don't need to escape `"""` (but you have to escape `"\"`) and vice versa.